

UNIT-3

-> **A tree** is a kind of non-linear data structure that is used to represent the data in hierarchical form.

-> **A search tree** is a tree data structure used for locating specific keys from within a set

-> **Binary search trees:** In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.

Advantages of Binary search tree

-> Searching an element in the Binary search tree is easy as we always have a hint that which subtree has the desired element.

-> As compared to array and linked lists, insertion and deletion operations are faster in BST.

-> **AVL trees:** AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.

-> Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

-> If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.

-> If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.

-> If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

-> AVL trees are **mostly used for in-memory sorts of sets and dictionaries**.

-> **WHY??** AVL tree controls the height of the binary search tree by not letting it to be skewed.

-> **B-tree:** B-tree is a special type of self-balancing search tree in which each node can contain more than one key and can have more than two children. It is a generalized form of the [binary search tree](#). It is also known as a height-balanced m-way tree.

-> We need for B-tree because it needs lesser time in accessing the physical storage media like a hard disk.

-> B tree is used to index the data.

-> **Red-Black** tree is a self-balancing binary search tree in which each node contains an extra bit for denoting the colour of the node, either red or black.

- **Property #1:** Red - Black Tree must be a Binary Search Tree.
- **Property #2:** The ROOT node must be coloured BLACK.
- **Property #3:** The children of Red coloured node must be coloured BLACK. (There should not be two consecutive RED nodes).
- **Property #4:** In all the paths of the tree, there should be same number of BLACK coloured nodes.
- **Property #5:** Every new node must be inserted with RED colour.
- **Property #6:** Every leaf (i.e. NULL node) must be coloured BLACK.

->splay trees: Splay trees are the self-adjusted binary search trees. It has one extra property that is splaying which makes it unique.

Splaying an element, is the process of bringing it to the root position by performing suitable rotation operations.

->There are six types of rotations used for splaying:

1. Zig rotation (Right rotation)
2. Zag rotation (Left rotation)
3. Zig zag (Zig followed by zag)
4. Zag zig (Zag followed by zig)
5. Zig zig (two right rotations)
6. Zag zag (two left rotations)

->2-3 tree: A **2-3 Tree** is a tree data structure where every node with children has either two children and one data element or three children and two data elements.

Properties of 2-3 tree:

- Nodes with two children are called 2-nodes. The 2-nodes have one data value and two children
- Nodes with three children are called 3-nodes. The 3-nodes have two data values and three children.
- Data is stored in sorted order.
- It is a balanced tree.
- All the leaf nodes are at same level.
- Each node can either be leaf, 2 node, or 3 node.
- Always insertion is done at leaf.

UNIT-4

->Heaps: A heap is a tree-based data structure in which (tree is a complete binary tree) all the nodes of the tree are in a specific order.

-> Heaps are used when the highest or lowest order/priority element needs to be removed.

Max-Heap: In a Max-Heap the key present at the root node must be greatest among the keys present at all of it's children. The same property must be recursively true for all sub-trees in that Binary Tree.

Min-Heap: In a Min-Heap the key present at the root node must be minimum among the keys present at all of it's children. The same property must be recursively true for all sub-trees in that Binary Tree

Heap Data Structure Applications

- >Heap is used while implementing a priority queue.
- >Dijkstra's Algorithm
- >Heap Sort

Heap Data Structure Operatins

-> **Heapify** is the process of creating a heap data structure from a binary tree. It is used to create a Min-Heap or a Max-Heap.

-> Insert Element into Heap

-> Delete Element from Heap

-> **Peek operation(Find max/min)** returns the maximum element from Max Heap or minimum element from Min Heap without deleting the node.

-> **Extract-Max/Min:** Extract-Max returns the node with maximum value after removing it from a Max Heap whereas Extract-Min returns the node with minimum after removing it from Min Heap.

-> **priority queue implementation:** A priority queue is a special type of queue in which each element is associated with a priority value.

-> Elements are served on the basis of their priority.

-> Higher priority elements are served first.

-> If elements with the same priority occur, they are served according to their order in the queue.

-> **Priority queue can be implemented** using an array, a linked list, a heap data structure, or a binary search tree. Among these data structures, heap data structure provides an efficient implementation of priority queues.

Priority Queue Applications

Some of the applications of a priority queue are:

- Dijkstra's algorithm
- for implementing stack
- for load balancing and interrupt handling in an operating system
- for data compression in Huffman code

-> **heap sort:** Heap sort is a comparison-based sorting technique based on Binary Heap data structure. It is similar to selection sort where we first find the minimum element and place the minimum element at the beginning. We repeat the same process for the remaining elements.

-> **binomial heaps:** A binomial heap can be defined as the collection of binomial trees that satisfies the heap properties, i.e., min-heap. The min-heap is a heap in which each node has a value lesser than the value of its child nodes. Mainly, Binomial heap is used to implement a priority queue.

A Binomial Tree is a recursively defined tree. A binomial tree of order 0 has 1 node. A Binomial Tree of order k can be constructed by taking two binomial trees of order $k-1$ and making one as leftmost child or other.

-> **Fibonacci heaps:** Fibonacci Heap is a collection of trees with min-heap or max-heap property. In a fibonacci heap, a node can have more than two children or no children at all. Also, it has more efficient heap operations than that supported by the binomial and binary heaps.

-> The fibonacci heap is called a **fibonacci heap** because the trees are constructed in a way such that a tree of order n has at least F_{n+2} nodes in it, where F_{n+2} is the $(n + 2)^{\text{th}}$ Fibonacci number.